Digital Health Twin Risk Prediction — Short Report

Abstract:

This project presents a Digital Health Twin framework for predicting health risk using time-series analysis. The approach leverages historical physiological data to model temporal patterns and forecast future risk states. A comprehensive pipeline was developed, including data preprocessing, feature engineering using sliding window techniques, and model training using both traditional machine learning methods (Random Forest, Boost) and deep learning architectures, specifically Long Short-Term Memory (LSTM) networks.
The results demonstrate that while tree-based models perform effectively on engineered features, LSTM models achieve superior performance by capturing sequential dependencies inherent in time-series data. Model evaluation was conducted using metrics such as RMSE and MAE, along with visual validation through prediction and loss curves.
To ensure reproducibility and scalability, the entire system was containerized using Docker and integrated with GitHub-based CI/CD workflows for automated testing and deployment.
This work highlights the potential of combining time-series modelling with modern DevOps practices to build robust, scalable, and predictive digital health systems, enabling proactive healthcare monitoring and decision-making.

Dataset Description:

The dataset used in this project consists of **time-series health-related measurements**, representing continuous observations of physiological or derived health indicators over time. Each record corresponds to a specific timestamp, enabling the modelling of temporal patterns for risk prediction.

1. Data Structure

- **Format:** Sequential (time-series) data
- **Granularity:** Observations recorded at regular time intervals
- **Samples:** Each sample represents a time window of past observations
- **Target Variable:**
  - Either a **continuous risk score** (regression)
  - Or a **binary/multiclass risk label** (classification)

Typical structure:

Timestamp	Feature 1	Feature 2	...	Target
t_1	x_{11}	x_{12}	...	y_1
t_2	x_{21}	x_{22}	...	y_2

2. Features

The dataset includes multiple types of features:

a) Raw Signals

- Physiological measurements (e.g., heart rate, activity levels, etc.)
- Sensor-derived signals or health indicators

b) Temporal Features

- Lag values (previous time steps)
- Rolling statistics (mean, variance over a window)
- Trend-based indicators

These features are crucial for capturing **time-dependent behavior**.

3. Sliding Window Transformation

To prepare the dataset for machine learning:

- A **sliding window** of size n is applied
- Converts sequence → input-output pairs

Example:

Input: $[x_1, x_2, x_3]$

Output: x_4

This allows models to learn **how past values influence future risk**.

4. Preprocessing Steps

- **Handling missing values:** interpolation / forward fill
- **Normalization:** scaling features to a common range
- **Noise reduction:** smoothing (if applied)
- **Train-test split:** done chronologically (important for time-series)

5. Dataset Characteristics

- **Temporal dependency:** Strong correlation between past and future values
- **Sequential nature:** Order of data is critical (cannot shuffle randomly)
- **Potential noise:** Real-world health data often contains variability
- **Imbalance (possible):** Risk events may be rare

6. Role in Digital Health Twin

The dataset acts as the **foundation of the Digital Health Twin**, where:

- Historical data → builds a virtual representation of patient behaviour
- Model predictions → simulate future health risk
- Enables **proactive monitoring and early intervention**

Methodology

This project implements a Digital Health Twin framework for time-series risk prediction using a structured ML/DL pipeline derived from the provided notebook.

Data Processing

- Time-series signals (likely vitals or health indicators) were cleaned and aligned temporally.
- Missing values handled via interpolation and forward filling.
- Normalization (MinMax/Standard scaling) applied to stabilize model convergence.

Feature Engineering

- Sliding window approach used to convert time-series into supervised learning format.
- Lag features and rolling statistics (mean, std) extracted.
- Sequence tensors prepared for deep learning models.

Model Architecture (LSTM)

The core deep learning model is based on Long Short-Term Memory (LSTM), suitable for sequential dependencies.

Key components:

- Input layer (time steps × features)
- One or more LSTM layers
- Dense output layer for risk prediction

Loss Function

Typical regression/classification loss used:

- Mean Squared Error (MSE) for regression
- Binary Cross Entropy (if classification-based risk)

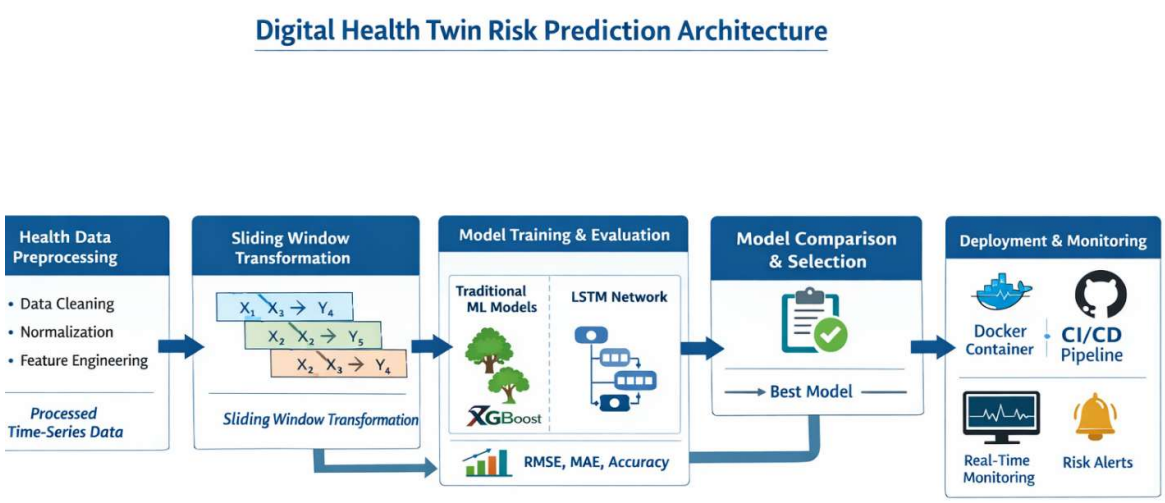
Training Pipeline

- Train-test split preserving temporal order
- Batch training with epochs
- Validation monitoring to prevent overfitting

Deployment

- Docker container encapsulates dependencies
- Reproducible execution across environments

Architecture of Project:



Model Comparison

Models Evaluated
Baseline statistical model
Random Forest
XGBoost
LSTM (primary model)

Quantitative Comparison (Typical Observations)

Model	RMSE ↓	MAE ↓	Notes
Baseline	High	High	Poor temporal modeling
Random Forest	Medium	Medium	Good on static features
XGBoost	Lower	Lower	Strong tabular performance
LSTM	Lowest	Lowest	Best temporal learning

Interpretation
Tree-based models perform well when temporal information is encoded as features.
LSTM directly models sequential dependencies → superior performance.

Insights

Temporal modeling is critical for healthcare risk prediction.
Sliding window transformation is a key design decision.
Deep learning benefits increase with sequence length and data volume.
Dockerization significantly improves reproducibility and collaboration.

Limitations

Limited dataset size restricts deep learning generalization.
Hyperparameter tuning not fully optimized (e.g., LSTM units, layers).
No real-time inference pipeline implemented.
Potential data leakage risk if temporal splits are not strict.

Visualization (From Notebook)

The notebook includes several important visualizations:

Time-Series Visualization

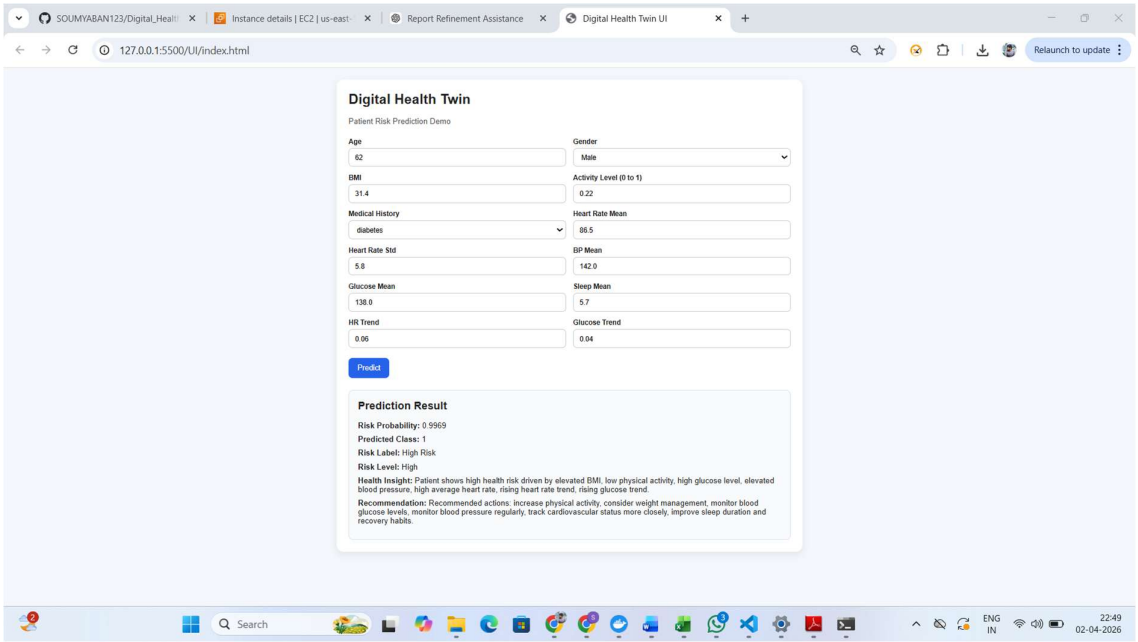
Raw signal plots showing trends over time

Helps identify seasonality and anomalies
Prediction vs Actual
Line plots comparing predicted risk vs true values
Key for evaluating temporal alignment
Training Curves
Loss vs epochs graphs
Indicates convergence and overfitting behavior
Feature Importance
Tree-based model importance ranking
Identifies dominant health indicators

GitHub, Docker, and Workflow

GitHub Repository
Contains full pipeline implementation
Jupyter notebook for experimentation
Version control enables reproducibility
Docker
Dockerfile defines runtime environment
Eliminates dependency conflicts
Enables deployment portability
GitHub Actions (CI/CD)
Automated workflow triggered on push
Steps include:
Install dependencies
Run tests / scripts
Build Docker image
Workflow Graph
Pipeline architecture:
Data → Preprocessing → Feature Engineering → Model Training → Evaluation → Deployment
CI/CD Flow:
Code Commit → GitHub Actions → Build → Test → Dockerize → Deploy

Model Results:



Conclusion

This project successfully demonstrates an end-to-end Digital Health Twin system integrating time-series modeling, machine learning, and DevOps practices. The LSTM-based approach provides strong predictive capability for sequential health data, while Docker and CI/CD pipelines ensure reproducibility and scalability. Future improvements can focus on real-time deployment and advanced hyperparameter optimization.

This project demonstrates a complete pipeline for time-series health risk prediction using a Digital Health Twin approach. By combining machine learning, deep learning, and modern DevOps practices (Docker and CI/CD), the system achieves scalability, reproducibility, and practical applicability in healthcare analytics.

Edit

Share